ChatGPT

Deep Technical Analysis of the Stage-2 Project

1. Overview

This repository is a heterogeneous bundle of scripts, template assets and specification documents aimed at solving two **very different problems**:

1. **Email safety pipeline** – three Markdown files (`stage1.md`, `stage2.md`, `stage3.md`) define the contract and logic for a multi-stage classification pipeline that evaluates the safety of incoming emails.
2. **Presentation tooling** – JavaScript and Python scripts (`answer.js`, `slides_template.js`, `create_montage.py`, `pptx_to_img.py`) and a collection of assets are used to generate slide decks and visual montages. These tools are unrelated to the email safety pipeline but share the same repository.

The coexistence of these two subsystems, without clear separation, introduces complexity and potential confusion. The following sections dissect each subsystem, map their dependencies and interactions, and identify weaknesses and recommendations for improvement.

2. Repository Structure

Below is a high-level inventory of the repository's contents. Only major files are listed; nested `node_modules` and similar vendor folders are omitted for brevity.

Path	Purpose / Notes
`answer.js`	Node script that uses `pptxgenjs` to create a PowerPoint presentation. Serves as a starting point for customizing slides.
`slides_template.js`	Rich example using `pptxgenjs` that defines multiple slide layouts (bar charts, timelines, images, bubble graphs). Intended as a design reference.
`create_montage.py`	Python script using Pillow to create a montage image from multiple PNGs. Used for visually inspecting slide outputs.
`pptx_to_img.py`	Python script that converts a PPTX to images and checks for content overflow by padding slides and rasterizing to PNG.
`placeholder_light_gray_block.png`	Placeholder asset used in slide examples.
`stage2.zip`	Archive containing the **email safety pipeline** specification files and example templates. Extracted into `stage2_unzipped/stage2`.

Path	Purpose / Notes
`node_modules/`, `package.json`	Standard Node.js dependency tree; `pptxgenjs` and `fontawesome` libraries are the core runtime dependencies.
`stage2_unzipped/stage2/stage1.md`	Stage-1 specification for the *Reputation Analyzer*. Defines input fields, output JSON schema and deterministic heuristics for flagging risky sender behaviour.
`stage2_unzipped/stage2/stage2.md`	Stage-2 specification for the *Content Analyzer*. Defines how to parse email HTML, extract signals, classify topics, apply dynamic thresholds and generate JSON output.
`stage2_unzipped/stage2/stage3.md`	Stage-3 specification for the *Combined Decision Analyzer*. Describes how to merge Stage-1 and Stage-2 results into a final decision with human-readable output.
`stage2_unzipped/stage2/пример шаблона*.txt`	Eight example email templates (Russian language) used to test the Stage-2 logic.
`stage2_unzipped/stage2/report.md`	Sample report summarizing how Stage-2 flagged each template.

3. Email Safety Pipeline

3.1 Stage-1: Reputation Analyzer

The Stage-1 component (`stage1.md`) evaluates the reputation of the sender based solely on metadata. Its key features are:

- **Fixed input schema** with six fields: `commentsFromSupport`, `audit`, `dataSource`, `sender_email`, `domain_name` and `strictness` (a numeric sensitivity parameter).
- **Normalization functions** such as `extractDomain`, `isKnownBrand`, and `isBrandlikePhishing`. The logic is deterministic and conservative.
- **Heuristic rules** that map specific patterns to risk levels (e.g., missing `dataSource` ⇒ red flag, presence of “bad” in `audit` ⇒ yellow).
- **Output** is STRICT JSON with a `verdict` (`red` | `yellow` | `green`) and a list of `reasons`. The `reasons` array uses uppercase key `FLAG`.

Potential issues / recommendations:

1. **Mixed case keys** – Stage-1 outputs `FLAG` in uppercase while Stage-2 uses lowercase `flag`. This inconsistency complicates the Stage-3 aggregator. Harmonising the schema across stages would reduce friction.
2. **Missing context for brand detection** – `isBrandlikePhishing` is described but not implemented. Without explicit logic, the detection threshold may be arbitrary. A data-driven approach (e.g., a whitelist/blacklist of domains and fuzzy matching) would improve reliability.

3. **Hard-coded domain checks** – `isKnownBrand` returns true only if the domain exactly matches a known brand. This approach misses obvious variations (e.g., `sberbank.ru` vs `sberbank.com`). Implementing fuzzy matching or using a list of known legitimate domains and their aliases would reduce false positives/negatives.
4. **Strictness parameter is underutilised** – Stage-1 logic only affects the phishing domain check. A more holistic use of `strictness`, such as adjusting thresholds for all heuristics, would give moderators finer control.

3.2 Stage-2: Content Analyzer

The Stage-2 component (`stage2.md`) examines the HTML and metadata of an email to identify phishing, scam and manipulative content. It expands the pipeline's scope beyond sender reputation to the **actual email body**.

Key functions and logic:

1. **Normalization & Pre-processing**
2. HTML tags are stripped and whitespace collapsed to get a clean text representation.
3. URLs are normalised and the visible anchor text is extracted via `getAllLinks`.
4. A default `strictness` value (0.5) is applied when not provided.
5. **Signal extraction**
6. Textual signals: `hasPhishingRequest`, `hasUrgentLanguage`, `hasScamPromise`, `hasAggressiveCTA`, `hasGenericGreeting`, `hasPoorGrammar`.
7. Structural signals: presence of unsubscribe links (`hasUnsubscribeLink`), contact information (`hasContactInfo`) and legal disclosures (`hasLegalese`).
8. Visual and tone analysis: `analyzeVisualStyle` returns `suspicious` or `ok` based on colours, fonts and image quality; `analyzeTone` classifies tone as `neutral`, `pressuring` or `scammy`.
9. Sender and subject checks: `isKnownBrandImitation`, `isMisleadingSenderName`, `isGenericEmail`, `isSuspiciousDomain`, `isAggressiveSubject`, `subjectMatchesContent`.
10. Attachment scanning: `hasMaliciousAttachments` looks for high-risk file types (.exe, .scr, .zip, .rar). **Note**: `.zip` appears in the malicious list; including `.zip` will flag nearly all zipped attachments, which may not be appropriate for legitimate archives.
11. **Topic classification**
12. `detectScamTopic` returns a `label`, `bucket` and `confidence`. High-risk buckets include threats like "security breaches" and "extortion"; ambiguous buckets cover financial products, gambling, medical miracles and easy earnings.
13. A **dynamic confidence threshold** is computed: `threshold = 0.9 − 0.5 × strictness`. With strictness=0.0 the threshold is 0.9 (only very confident topics trigger). With strictness=1.0 the threshold drops to 0.4, making the filter more sensitive.
14. **Aggregation & verdict determination**

15. The number of red and yellow harm signals is counted; legitimate signals are also counted but the specifics are unclear.
16. If a high-risk topic is detected and at least one red harm signal exists, the **verdict** is immediately "red". For ambiguous topics, the presence of harm signals will typically upgrade the verdict to at least "yellow".
17. The final JSON includes the `verdict`, the incorrectly spelled field `anazyle` (analysis summary), the `topic` (or null) and a list of `reasons` (lowercase `flag`).

Strengths: * The Stage-2 design is more comprehensive than Stage-1, combining textual, visual and metadata signals. * The dynamic threshold is a clever way to adjust sensitivity based on a `strictness` input, making the system adaptable to different risk tolerances.

Weaknesses / conflicts:

1. **Naming inconsistency** – `anazyle` is a misspelling of "analyse" or "analysis". This typo will propagate downstream into Stage-3. It should be corrected before implementation.
2. **Lowercase** `flag` **vs uppercase** `FLAG` – as mentioned above, Stage-2 uses lowercase `flag` while Stage-1 uses uppercase. Stage-3 must handle both; this is error-prone and inconsistent. Unifying the field names would simplify integration.
3. **Attachment misclassification** – listing `.zip` and `.rar` as malicious by default may be too aggressive. Many legitimate businesses send zipped documents (e.g., invoices). Consider differentiating by size, password protection, or scanning the contents rather than blanket banning.
4. **Tone and visual analysis** – the specification defines `analyzeVisualStyle` and `analyzeTone` but does not prescribe concrete algorithms. Without objective criteria, these functions risk subjective judgements or inconsistent outputs. A future version should define quantitative metrics (e.g., colour histogram analysis, lexical stress detection) or integrate pre-trained models.
5. **Lack of multilingual support** – although the example templates are in Russian, the helper functions are defined in English. For reliable real-world deployment, language detection and locale-specific phrase lists are necessary.
6. **Ambiguous bucket handling** – the specification notes that ambiguous topics require "co-evidence" but does not define how strong that evidence must be. The logic in the pseudo-code is incomplete in this area.

3.3 Stage-3: Combined Decision Analyzer

Stage-3 (`stage3.md`) synthesises the outputs from Stage-1 and Stage-2 and generates a final human-readable report. It is essentially a rule-based aggregator with the following logic:

1. **Combine reasons and flag counts** – all reasons from Stage-1 and Stage-2 are merged. The number of red and yellow flags is counted regardless of origin.
2. **Decision rules** – a `STOP` decision (block email) is issued if:
3. Stage-1 verdict is red; or
4. Any red flags exist; or
5. Both Stage-1 and Stage-2 verdicts are yellow; or
6. The total yellow flags exceed a configurable threshold (`yellow_flag_threshold` defaults to 3). Otherwise the status is `GO`.
7. **Flag summary** – if any red flags exist, `FLAGS` is `RED(n)`, otherwise if any yellow flags exist, `FLAGS` is `YELLOW(n)`, else `GREEN`.

8. **Human-readable output** – a plain text string is assembled with four fields: `STATUS`, `FLAGS`, `REASON` (a bulleted list of all comments) and `ANALYZE`, a free-form explanation of the decision.

Observations and recommendations:

1. **Asymmetric severity** – Stage-3 does not allow a green Stage-2 verdict to override a yellow Stage-1 verdict. Given that Stage-1 only checks reputation, a yellow Stage-1 verdict (e.g., public email domain) may be benign. Consider allowing Stage-2 to "downgrade" the severity if the email content is clearly safe and legitimate signals are high.
2. **Yellow threshold logic** – the default threshold of 3 yellow flags may be too low or too high depending on the distribution of flags. It should be tuned on real-world data. Exposing the threshold as configuration (already present) is a good design choice; ensure that external systems can modify it.
3. **Ordering of reasons** – Stage-3 concatenates reasons from Stage-1 and Stage-2 in the order they appear, but does not deduplicate duplicate comments. There is a risk of repeated or contradictory messages. Implement a deduplication step similar to Stage-1's `seen` set to avoid cluttering the moderator report.
4. **Format rigidity** – the required output format is extremely rigid. If future fields need to be added (e.g., probability scores), the Stage-3 spec would need revision. Consider using a more flexible JSON output instead of plain text.

3.4 Example Templates and Report

The extracted `пример шаблона*.txt` files illustrate the kinds of emails the system is meant to detect. They range from harmless order confirmations to blatant scam advertisements. The `report.md` summarises how Stage-2 judged these samples. In general, the classification appears reasonable:

Template	Stage-2 verdict	Notable signals
шаблон1 (Фотостудия)	green	Contains unsubscribe link and contact info; no urgency or scam.
шаблон2 (Биржа)	yellow	Urgent language, financial topic.
шаблон3 (Казино)	red	Strong urgency, aggressive CTA, gambling topic, scam promises.
шаблон4–7 (Кредиты)	red/yellow	Unrealistic loans (0 % interest), shortened links, missing contacts.
шаблон8 (Заказ)	green	Standard order confirmation with phone and unsubscribe links.

The report helps validate the heuristics but also reveals limitations (e.g., all financial promotions with unrealistic promises are immediately flagged red, even if they come from legitimate promotions). A future iteration could cross-reference reputation and regulatory disclosures to better distinguish legitimate financial offers from scams.

4. Presentation Tooling

The repository also contains scripts and assets for generating **PowerPoint presentations** using JavaScript (`pptxgenjs`) and Python (Pillow, python-pptx). While these are not directly tied to the email safety pipeline, they are present in the same project and merit analysis.

4.1 `answer.js`

This Node script demonstrates how to use the `pptxgenjs` library to create a slide deck. It defines constants for slide dimensions, font sizes, colours, and helper functions (`calcTextBoxHeight`, `imageSizingContain`, `imageSizingCrop`). The script currently generates a single slide with a placeholder image and textual placeholders. Additional slides would need to be added manually.

Observations:

1. **Hard-coded placeholders** – the script includes placeholder text ("Presentation title", "Subtitle here") and uses a light-gray block image. For production use, these should be parameterised or replaced with real assets.
2. **Asynchronous I/O** – the main function is asynchronous but does not handle potential errors from `pptx.writeFile`. Surrounding the call with `try/catch` would allow error reporting.
3. **Module coupling** – many constants (font sizes, colours) are duplicated in both `answer.js` and `slides_template.js`. Extracting these into a shared module would prevent divergence and ease maintenance.

4.2 `slides_template.js`

This file is a detailed showcase of advanced slide layouts using `pptxgenjs`. It demonstrates how to create bar charts, timelines, image grids, bubble graphs and tables. Each slide is fully scripted.

Strengths:

- Rich variety of layouts demonstrates the capabilities of `pptxgenjs` and can serve as a reference for custom slide creation.
- Helper functions like `addSlideTitle` and `imageSizingContain/crop` encapsulate common layout tasks.

Weaknesses:

1. **No separation between data and layout** – the slide contents (titles, labels, chart values) are hard-coded. To create dynamic presentations, these should be parameterised or loaded from external data sources.
2. **Large monolithic script** – the entire template is contained in a single self-invoking async function. Breaking it into reusable components (e.g., separate functions for each slide type) would improve readability and testability.
3. **Lack of error handling** – there is no error checking when adding images (e.g., missing files) or charts. A try/catch wrapper would surface issues during generation.

4.3 Python utilities

- `create_montage.py` combines multiple PNG images into a single montage image. It uses `PIL.Image` and resizes the final montage if it exceeds a maximum dimension. The code is straightforward and efficient.
- `pptx_to_img.py` converts a PPTX to images, enlarges the deck with padding and checks for overflow (i.e., content that spills outside the slide boundaries). It leverages LibreOffice to convert PPTX → PDF, then uses `pdf2image` to rasterise the slides. It also uses `numpy` to compare pixel colours at the margins.

Potential issues:

1. **Dependence on LibreOffice** – the script calls `soffice` via `subprocess.run`. This hard dependency is brittle and may fail if LibreOffice is not installed or not in the PATH. A Dockerised environment or direct use of `python-pptx` and `PIL` could avoid external binaries.
2. **Runtime performance** – rasterising high-DPI images of PPT slides is computationally expensive. For large decks, the `convert_from_path` call may consume significant memory. Caching or parallelising the conversion could help.
3. **Padding mismatch tolerance** – the logic for detecting overflow is based on a tolerance parameter derived from DPI. Empirically verifying these thresholds on a variety of slide designs would be advisable.

5. Cross-Subsystem Considerations

While the email safety pipeline and presentation tooling are separate concerns, housing them in one repository invites confusion. Here are some cross-cutting observations:

1. **Lack of modularity** – Both subsystems share global constants and duplicate code (e.g., font sizes in `answer.js` and `slides_template.js`). Extracting shared definitions into separate modules (for Node scripts) or packages (for Python) would reduce duplication.
2. **Inconsistent language and naming** – Mixed Russian/English naming (e.g., `пример шаблона`), the misspelled `anazyle` field, and inconsistent case (`flag` vs `FLAG`) point to poor quality control. Standardising naming conventions across all files would improve maintainability.
3. **Unclear deliverables** – Without clear separation, it is unclear whether the final goal is to build a slide deck summarising the email safety pipeline or to implement the pipeline itself. Creating separate directories (e.g., `pipeline/` and `presentation/`) or separate repositories would clarify the project scope.

6. Recommendations

1. **Unify JSON schemas** – Adopt a consistent schema for reasons across all stages (`field`, `flag`, `comment`). Avoid uppercase/lowercase mismatches and correct typos (change `anazyle` to `analysis`).
2. **Parameterise heuristics** – Encapsulate the helper functions defined in the Markdown specs into actual code modules with unit tests. This includes implementing `detectScamTopic`, `isBrandlikePhishing`, `analyzeVisualStyle` and `analyzeTone` using well-defined algorithms or machine-learning models.
3. **Integrate strictness consistently** – Use the `strictness` parameter not just for domain phishing checks but across all heuristics (e.g., adjusting thresholds for urgent language

detection or scam promises). Document how strictness influences the dynamic confidence threshold and final decisions.

4. **Improve attachment analysis** – Don't blanket-block `.zip` or `.rar` attachments. Instead, extract their contents in a sandbox and scan for executables or macros. Use virus scanning tools or heuristic file type detection.
5. **Enhance multilingual support** – Expand the phrase lists for `hasUrgentLanguage`, `hasScamPromise`, etc., to include multiple languages, and perform language detection before applying heuristics. This prevents misclassification of foreign-language emails.
6. **Separate concerns** – Split the repository into two clearly defined projects:
7. **Email-safety pipeline** – containing the Stage-1/2/3 specifications, helper function implementations, unit tests and integration tests with sample templates.
8. **Presentation toolkit** – containing `answer.js`, `slides_template.js`, Python utilities and assets. Provide clear instructions on how to generate slides summarising the pipeline's analysis or any other data.
9. **Improve documentation and comments** – Convert the Markdown specs into proper design documents or README files that explain the intended flow, provide example inputs/outputs and clarify ambiguous areas (e.g., what constitutes a "generic greeting" or "suspicious visual style"). Inline comments in code should explain non-obvious logic and decisions.
10. **Adopt version control practices** – Use semantic versioning or git branches to track changes to the pipeline logic separately from the presentation code. This will help prevent accidental regressions and make it easier to roll back.

7. Conclusion

This project is ambitious in scope: it combines a rule-based email safety analyser with a slide-deck generator. Each subsystem has merit on its own, but their cohabitation in a single repository, coupled with inconsistent naming and incomplete implementations, leads to architectural friction and potential bugs. By unifying schemas, parameterising heuristics, separating concerns and improving documentation, the project can mature into a robust solution that helps both machines and human moderators detect and communicate risks in inbound emails. Ignoring these structural issues will likely result in maintenance headaches, misclassifications and confusion among developers and stakeholders.
